

Incident Intelligence Platform

From a simple CRUD API to an AI-powered, event-driven, data-engineered, cloud-deployed incident management platform: 27 phases delivered in 9 versions, each with a defined backend and frontend output.

Project Name	Incident Intelligence Platform
Project Type	Backend + Data Engineering + Frontend Roadmap / Portfolio Build
Core Stack	ASP.NET Core, EF Core, SQL Server, Redis, RabbitMQ, Kafka, Spark, Airflow, LLM/RAG, Angular + TypeScript, Docker, Azure
Delivery Model	27 Progressive Phases grouped into 9 Versions (MVP, V2 to V9), milestone-based
Document Purpose	Phases, scope, and expected backend and frontend output per phase and per version

What changed in this update

- The original 10 phases are kept (Phases 1 to 10). 17 new phases (11 to 27) are added: reliability, database engineering, Kafka, data lake, Spark, warehouse, Airflow, data quality, analytics, advanced RAG, security, testing, load testing, Docker/CI/CD, cloud, and the final production architecture.
- Every phase now has an explicit Backend Output and Frontend Output. The original proposal described only the backend, so the frontend (Angular + TypeScript, paired with the .NET backend) is a new addition.
- Phases are grouped into 9 versions. Each version is a releasable, demonstrable product state, with a demo scenario.
- Status: Phases 1 to 4 are marked Completed and Phase 5 In Progress, based on your note that you are currently at Phase 5. Adjust if that is not accurate.
- Small corrections: Resolve Incident endpoint added to Phase 1 (it was in the output flow but not in the endpoint list); the Phase 4 transition rule is stated once: Open > Resolved is rejected unless a rule explicitly allows it; Deployment ingestion added to Phase 5 because the correlation rule depends on it.

Guiding principle (unchanged): build only what the current phase needs. A phase is complete when its outputs work end to end, not when a deadline passes. Phase 1 had to be finished before any later phase began, and each version builds on the previous one without rewriting it.

Version Roadmap at a Glance

Version	Name	Phases	Release result
MVP	Incident Tracker	1 to 4	A secure, usable incident management application with history and enforced workflow.
V2	Detection & Performance	5 to 7	The platform detects incidents on its own and serves dashboards quickly.
V3	Event-Driven, AI & Observable	8 to 10	Decoupled processing, AI root cause analysis, and production-grade observability.
V4	Resilient & Database-Hardened	11 to 12	The system survives failures and scales its data layer deliberately.
V5	Streaming & Data Lake	13 to 15	Kafka becomes the event backbone and a raw/clean/curated data lake begins to fill.
V6	Processing, Warehouse & Orchestration	16 to 18	Large-scale processing and a dimensional warehouse, run automatically on schedule.
V7	Trusted Data, Analytics & Advanced AI	19 to 21	Quality-checked data powers SRE metrics and history-aware AI.
V8	Secure, Tested & Proven	22 to 24	The platform is hardened, fully tested, and load-validated.
V9	Production Release (1.0)	25 to 27	Containerized, deployed to the cloud, and documented as a complete production architecture.

Phase Index

#	Phase	Version	Status
1	Core Incident API	MVP	Completed
2	Professional API	MVP	Completed
3	Authentication & Authorization	MVP	Completed
4	Incident Timeline & Domain Logic	MVP	Completed
5	Logs & Correlation	V2	In Progress
6	Background Processing	V2	Planned
7	Redis & Performance	V2	Planned
8	Event-Driven Architecture (RabbitMQ)	V3	Planned
9	AI Root Cause Analysis	V3	Planned
10	Observability & Production Readiness	V3	Planned
11	Advanced Reliability	V4	Planned
12	Advanced Database Engineering	V4	Planned
13	Kafka Event Streaming	V5	Planned
14	Event Streaming Pipeline	V5	Planned
15	Data Lake / Data Storage Layer	V5	Planned
16	Spark Data Processing	V6	Planned
17	Data Warehouse	V6	Planned
18	Airflow / Data Orchestration	V6	Planned
19	Data Quality	V7	Planned
20	Analytics Layer	V7	Planned
21	Advanced AI / RAG	V7	Planned

#	Phase	Version	Status
22	Security Hardening	V8	Planned
23	Testing Strategy	V8	Planned
24	Performance & Load Testing	V8	Planned
25	Docker & Deployment	V9	Planned
26	Cloud	V9	Planned
27	Production Architecture	V9	Planned

Technology Stack by Layer

Layer	Technologies	Introduced
Backend API	C#, ASP.NET Core Web API, EF Core, FluentValidation, Serilog, JWT/Identity	MVP
Operational data	SQL Server, Redis (cache-aside)	MVP, V2
Async and messaging	Hangfire, RabbitMQ, SignalR, Outbox pattern	V2, V3
AI	LLM API, RAG, embeddings, vector store (pgvector/Qdrant/Azure AI Search)	V3, V7
Observability	OpenTelemetry, Jaeger/Grafana, health checks	V3
Reliability and DB	Polly, Toxiproxy, execution plans, indexing, concurrency control	V4
Streaming and data	Kafka, Parquet data lake (MinIO), PySpark, warehouse (star schema), Airflow, Great Expectations/Soda	V5 to V7
Frontend	Angular (CLI, standalone components, Angular Material, RxJS, signals), TypeScript, Chart.js/ngx-charts, @microsoft/signalr, Jest/Karma, Playwright	MVP to V9
Quality and delivery	Testcontainers, k6, Docker, Docker Compose, GitHub Actions/Azure DevOps	V3, V8, V9
Cloud	Azure (Container Apps/AKS, Azure SQL, Redis, Key Vault, Monitor), Bicep/Terraform	V9

Frontend note: the original proposal had no frontend. Phases 1 to 3 use the UI mainly to prove the API; from Phase 4 onward the UI becomes the primary demonstration of each phase's output. The frontend is intentionally thin where the work is data or infrastructure (Phases 13 to 19), where its role is admin visibility rather than a product feature.

MVP: Incident Tracker

Phases 1 to 4

Version goal: A secure, usable incident management application with history and enforced workflow.

Backend release

Authenticated REST API with layered architecture, validation, error handling, roles, state machine, and incident timeline.

Frontend release

Angular app with login, role-aware incident list (paged, filtered), create/edit forms, incident detail page with timeline and status stepper.

Demo scenario: Developer logs in, creates an incident, moves it Open > Investigating > Mitigated; IncidentManager resolves it; Viewer can only read.

Phase 1: Core Incident API

COMPLETED

Goal: Build the first functional backend: Service and Incident CRUD.

BACKEND OUTPUT

- Entities: Service; Incident (Id, Title, Description, Severity, Status, CreatedAt, ResolvedAt, ServiceId). One Service has many Incidents.
- Endpoints: POST/GET /api/services, GET /api/services/{id}; POST/GET /api/incidents, GET/PUT/DELETE /api/incidents/{id}.
- Added: POST /api/incidents/{id}/resolve (Resolve was in the original output flow but missing from the endpoint list).
- EF Core migrations on SQL Server, DI, DTOs, correct HTTP status codes, Swagger.

FRONTEND OUTPUT

- No product UI yet: Swagger UI is the client for the flow create Service > create Incident > retrieve > update > resolve.
- Scaffold the Angular app (Angular CLI, standalone components, TypeScript, Angular Material), routing shell, and a typed API client generated from the OpenAPI spec (NSwag / OpenAPI Generator).
- Two unstyled read-only pages (Services list, Incidents list) to prove the API contract.

Key skills: ASP.NET Core Web API, EF Core, SQL Server, DI, LINQ, DTOs, HTTP status codes.

Done when: The full Swagger flow works end to end; the scaffolded UI lists real data.

Phase 2: Professional API

COMPLETED

Goal: Turn the CRUD API into a professionally structured backend.

BACKEND OUTPUT

- Layered flow: Controller > Service > Repository/EF Core > Database. No raw entities exposed.
- Request/Response models: CreateIncidentRequest, UpdateIncidentRequest, IncidentResponse.
- Validation (FluentValidation): Title required, Description max length, Severity valid, ServiceId must exist.
- Global exception handling with one consistent error envelope (400/401/403/404/409/500) and Serilog structured logging.
- Pagination, filtering, sorting: GET /api/incidents?page=1&pageSize=20&severity=High&status=Open&sort=createdAt:desc returning { items, page, pageSize, totalCount }.

FRONTEND OUTPUT

- Incidents table with server-side pagination, sorting, and Severity/Status filters (state kept in the URL).
- Create/Edit Incident forms showing inline field errors mapped from the API validation response.
- Global error handling: toasts for 404/409/500; loading, empty, and error states; severity and status badges.
- Service dropdown populated from the API.

Key skills: DTOs, validation, middleware, Serilog, pagination, filtering, consistent API responses.

Done when: Every endpoint returns the standard envelope; the UI paginates and filters without client-side hacks.

Phase 3: Authentication & Authorization		COMPLETED
Goal: Turn the system into a secure multi-user platform.		
BACKEND OUTPUT	FRONTEND OUTPUT	
• Roles: Admin, Developer, IncidentManager, Viewer. • POST /api/auth/register, /login, /refresh (plus /logout to revoke the refresh token). • JWT access token (short expiry) + refresh token (stored hashed); password hashing; token expiration. • Role-based and policy-based authorization on every endpoint: Viewer = read-only; Developer = create/update; IncidentManager = resolve; Admin = full access + user/role management. • Correct 401 (not logged in) vs 403 (not allowed) responses.	• Login and Register pages; AuthService with signals/RxJS state; route guards (CanActivate) on protected routes. • HTTP interceptor attaches the JWT and performs silent token refresh on 401 with request retry; logout. • Role-aware UI: actions hidden/disabled by role (for example, Resolve only for IncidentManager/Admin). • Admin Users page to assign roles; dedicated 401/403 screens.	
Key skills: JWT, refresh tokens, RBAC, policy-based authorization, endpoint protection. Done when: Login > JWT > Role > Authorization > Incident API works, and the UI reflects the logged-in role.		

Phase 4: Incident Timeline & Domain Logic		COMPLETED
Goal: Make Incident a real domain object with history and enforced rules.		
BACKEND OUTPUT	FRONTEND OUTPUT	
• New entity IncidentEvent: IncidentCreated, SeverityChanged, StatusChanged, DeploymentDetected, ErrorSpikeDetected, Assigned, Resolved. • GET /api/incidents/{id}/timeline; POST /api/incidents/{id}/events; POST /api/incidents/{id}/transitions. • State machine Open > Investigating > Mitigated > Resolved. Invalid transitions (e.g. Open > Resolved, Resolved > Investigating) are rejected with 409 unless a rule explicitly allows them. • Every change writes a timeline event automatically (who, when, old value, new value) as an audit trail. • Assignee on Incident; IncidentResponse includes the list of allowed next transitions.	• Incident Detail page: status, severity, service, assignee, timestamps. • Vertical timeline component (chronological, icon per event type, actor and time). • Status stepper offering only the allowed next transitions returned by the API, with a confirmation dialog. • Add note/event form, assignee picker, inline message when a transition is rejected.	
Key skills: Domain logic, state machines, business rules, audit history, entity relationships. Done when: Opening an incident shows status, severity, and a complete timeline; illegal transitions are impossible in the UI and rejected by the API.		

V2: Detection & Performance

Phases 5 to 7

Version goal: The platform detects incidents on its own and serves dashboards quickly.

Backend release

Log and deployment ingestion, correlation rules, Hangfire detection jobs, Redis caching, statistics endpoint, optimized queries.

Frontend release

Logs Explorer, related logs and correlation banner on incidents, service health, admin jobs and rules pages, KPI dashboard.

Demo scenario: Post error logs after a deployment; an auto-detected incident appears with its logs; dashboard KPIs update.

Phase 5: Logs & Correlation

IN PROGRESS

Goal: Start detecting incidents from real operational data. No AI yet.

BACKEND OUTPUT

- New entity Log (Id, Timestamp, Level, Message, ServiceId, TraceId) with indexes on (ServiceId, Timestamp).
- POST /api/logs (single and batch), GET /api/logs (filters: service, level, from/to, traceId, paging).
- Deployment entity + POST/GET /api/deployments (required by the correlation rule).
- GET /api/incidents/{id}/logs: logs from the incident's service in the window around the incident.
- Correlation rules: Deployment + error-rate increase = possible correlation (writes DeploymentDetected / ErrorSpikeDetected timeline events); 5xx count above threshold = automatically create an Incident.

FRONTEND OUTPUT

- Logs Explorer: service, level, time-range, and TraceId filters; level color coding; click a TraceId to filter by it.
- Incident Detail gets a Related Logs tab and a mini error-rate chart with deployment markers.
- 'Possible correlation' banner linking to the suspected deployment.
- Simple Deployments list and form.

Key skills: Querying, indexing, correlation rules, entity relationships Incident > Service > Logs.

Done when: Posting error logs after a deployment creates an incident that already carries its logs, timeline, and service.

Phase 6: Background Processing

PLANNED

Goal: Move heavy processing out of the HTTP request/response cycle.

BACKEND OUTPUT

- Hangfire with SQL Server storage; log ingestion enqueues an analysis job and returns immediately.
- Recurring job AnalyzeServices() every 5 minutes: error rate, latency, existing incidents, and health per service.
- De-duplication so one service/rule never creates multiple open incidents; auto-created incidents tagged Source = System.
- Retry policy, failed-job handling, scheduled jobs, job history; Hangfire dashboard restricted to Admin.
- GET /api/services/{id}/health.

FRONTEND OUTPUT

- 'Auto-detected' badge and a Source filter in the incident list.
- Service health indicator (green/amber/red) on the Services page.
- Admin Background Jobs page: recent runs, failures, retry action (or embedded Hangfire dashboard).
- Admin Detection Rules page (thresholds, time window); new-incident indicator via periodic refresh.

Key skills: Hangfire, async processing, recurring and scheduled jobs, retries, job history.

Done when: Incidents appear automatically without anyone creating them, and failed jobs are visible and retried.

Phase 7: Redis & Performance		PLANNED
Goal: Make the system faster and ready for higher load.		
BACKEND OUTPUT		FRONTEND OUTPUT
• Redis cache-aside on GET /api/services and GET /api/incidents/statistics. • Dashboard statistics: Open Incidents, Critical Incidents, Resolved Today, Most Affected Service. • TTL strategy, cache invalidation on incident create/resolve, JSON serialization, consistent key naming. • Database optimization: indexes, query tuning, projection, AsNoTracking, no N+1 queries. • Fallback to SQL Server on cache miss; X-Cache: HIT/MISS header for debugging.		• Dashboard page: KPI cards (Open, Critical, Resolved Today, Most Affected Service), incidents-by-severity chart, trend chart. • Skeleton loaders, 'last updated' timestamp, auto-refresh. • Client-side caching in the data service (RxJS shareReplay with expiry) aligned to the server TTL. • KPI cards click through to a pre-filtered incident list.
Key skills: Redis, cache-aside, TTL, invalidation, serialization, indexes, efficient LINQ.		
Done when: The dashboard loads from cache with measurably lower latency, and caches invalidate correctly after writes.		

V3: Event-Driven, AI & Observable		Phases 8 to 10
Version goal: Decoupled processing, AI root cause analysis, and production-grade observability.		
Backend release		Frontend release
RabbitMQ with idempotent workers, outbox, SignalR, LLM/RAG analysis service, OpenTelemetry, health checks, Docker, integration tests.		Live real-time incident feed, messaging admin page, AI Root Cause panel with feedback, trace IDs in errors, system health page.
Demo scenario: A log arrives, a worker creates an incident, the UI updates live, the AI explains the cause, and the trace is viewable end to end.		

Phase 8: Event-Driven Architecture (RabbitMQ)		PLANNED
Goal: Decouple the API from asynchronous processing.		
BACKEND OUTPUT		FRONTEND OUTPUT
• POST /api/logs > save > publish LogReceived > RabbitMQ > Analysis Worker (separate .NET Worker Service) > analyze > create Incident. • Events: LogReceived, IncidentCreated, IncidentResolved, DeploymentDetected; exchanges, queues, and routing keys defined. • Manual acknowledgement, delayed retry, Dead Letter Queue. • Idempotent consumer: a ProcessedMessages table (unique MessageId) makes duplicate deliveries harmless. • Outbox pattern for reliable publishing; SignalR hub pushes IncidentCreated/Resolved to clients.		• Live updates over SignalR: new and resolved incidents appear without refresh, with toasts. • Log ingestion UI shows 'accepted' (HTTP 202) and processing status. • Admin Messaging page: queue depth, DLQ count, requeue action for dead-lettered messages. • Connection-status indicator with automatic reconnect.
Key skills: Producers, consumers, exchanges, ack/retry, DLQ, idempotency, eventual consistency.		
Done when: Delivering the same event twice produces exactly one incident, and the API never waits for analysis.		

Phase 9: AI Root Cause Analysis		PLANNED
Goal: Use structured operational data to help engineers find root causes.		
BACKEND OUTPUT • Input: Incident + relevant logs + recent deployments + timeline + events, assembled by a prompt builder with a token budget (log sampling/summarization). • Provider-agnostic LLM client first; then RAG that retrieves relevant logs/events before analysis. • Structured output stored in IncidentAnalysis: RootCause, Evidence[], Confidence, RecommendedActions[] (versioned per run). • POST /api/incidents/{id}/analysis (async, 202) and GET /api/incidents/{id}/analysis; POST .../analysis/feedback. • Guardrails: JSON-schema validation; every evidence item must reference a real log or event ID.		FRONTEND OUTPUT • 'AI Root Cause' panel: root cause headline, evidence list (each item links to its log/timeline entry), confidence meter, numbered recommended actions. • Generate/Regenerate button with progress state (analysis runs asynchronously). • 'AI-generated, verify before acting' notice; thumbs up/down with comment; analysis history.
Key skills: LLM APIs, prompt engineering, RAG basics, structured outputs, guardrails. Done when: A real incident gets a report with evidence, confidence, and actions, and engineers can rate it.		

Phase 10: Observability & Production Readiness		PLANNED
Goal: Turn the project into a production-like system.		
BACKEND OUTPUT • OpenTelemetry for logs, metrics, and traces (OTLP export). • Correlation/Request/Trace ID propagated Client > API > RabbitMQ > Worker > SQL Server. • Health checks (/health/live, /health/ready covering SQL, Redis, RabbitMQ); rate limiting (429 + Retry-After); Polly retry policies. • Serilog enriched with Traceld; Dockerfiles for API and Worker. • Integration tests (WebApplicationFactory + Testcontainers) and API tests.		FRONTEND OUTPUT • Every error toast shows the Request/Trace ID with a copy button; frontend sends X-Correlation-Id. • 'View trace' link to Jaeger/Grafana Tempo for Admin/Developer. • System Health page: live/ready status of API, SQL, Redis, RabbitMQ. • Friendly 429 handling in the HTTP interceptor; Playwright smoke tests.
Key skills: OpenTelemetry, health checks, rate limiting, Docker, integration testing. Done when: A single failing request can be traced from the browser to the database and back.		

V4: Resilient & Database-Hardened		Phases 11 to 12
Version goal: The system survives failures and scales its data layer deliberately.		
Backend release Circuit breakers, retries, bulkheads, idempotency keys, graceful degradation, failure drills, execution-plan-driven indexing, concurrency control, 1M-row benchmarks.		Frontend release Degraded-mode banners, safe retries, concurrency conflict dialog, virtual-scroll infinite logs, performance report page.
Demo scenario: Kill Redis, RabbitMQ, and the LLM in turn: the UI shows degraded states and recovers with no data loss.		

Phase 11: Advanced Reliability	PLANNED
Goal: Make the system resilient to failures.	
BACKEND OUTPUT • Timeouts on all outbound calls; retries with exponential backoff and jitter; circuit breakers; bulkhead isolation (separate pools for ingestion, queries, AI). • Rate limiting per user/IP/service key; Idempotency-Key support on POST endpoints. • Graceful degradation: Redis down = serve from SQL; LLM down = queue analysis and return 'pending'; RabbitMQ down = outbox buffers events. • Failure drills (e.g. with Toxiproxy): SQL Server down, Redis down, RabbitMQ down, worker crash, LLM API down, network failure, duplicate event, slow database. • Resilience report: expected vs observed behavior per scenario.	FRONTEND OUTPUT • Degraded-mode banners ('Statistics may be stale', 'AI analysis temporarily unavailable'). • Retry buttons with backoff; double-submit protection with an Idempotency-Key generated per form submit. • Offline / connection-lost state. • System Status page extended with circuit-breaker state (Admin).
Key skills: Timeout, retry, backoff, circuit breaker, bulkhead, idempotency, graceful degradation. Done when: Each failure scenario has a documented, tested behavior and the UI stays usable during it.	

Phase 12: Advanced Database Engineering	PLANNED
Goal: Understand what happens underneath EF Core.	
BACKEND OUTPUT • Query execution plan analysis for log search, incident filtering, and statistics. • Index design: composite (ServiceId, Timestamp), covering (INCLUDE), filtered indexes. • Transactions, isolation levels (READ COMMITTED SNAPSHOT), locks; reproduce and fix a deadlock. • Optimistic concurrency (rowversion) on Incident; pessimistic locking where justified; connection pooling; constraints (FK, CHECK, UNIQUE). • Keyset pagination for logs; benchmarks at 10K / 100K / 1M logs comparing query time, CPU, memory, and index usage, with before/after report.	FRONTEND OUTPUT • 409 concurrency dialog: 'Someone else updated this incident' with reload/compare. • Logs Explorer upgraded to keyset infinite scroll with Angular CDK virtual scroll, fast at 1M rows. • Admin Query Performance page: charts of query time vs row count and before/after index results.
Key skills: Execution plans, indexing, isolation levels, deadlocks, concurrency control, benchmarking. Done when: Benchmark report shows measurable gains, and concurrent edits are handled safely end to end.	

V5: Streaming & Data Lake

Phases 13 to 15

Version goal: Kafka becomes the event backbone and a raw/clean/curated data lake begins to fill.

Backend release

Kafka topics, producers, four independent consumer groups, notification service, Parquet data lake with catalog.

Frontend release

Streaming admin page, live event tail, notification center and preferences, data catalog and dataset preview.

Demo scenario: One IncidentCreated event triggers incident handling, analytics, a Slack/email notification, and a lake write independently.

Phase 13: Kafka Event Streaming

PLANNED

Goal: Move from messaging to a real event-streaming architecture.

BACKEND OUTPUT

- Kafka (KRaft) cluster; topics for incidents, logs, deployments, and service health, partitioned by ServiceId so events stay ordered per service.
- Producer with acks=all and idempotence; events: IncidentCreated, IncidentResolved, LogReceived, DeploymentDetected, ServiceDegraded.
- Consumer groups, offset commit strategy, replication, retention, ordering, delivery semantics (at-least-once + idempotent consumers), consumer-lag monitoring.
- Versioned event schemas (JSON Schema/Avro).
- Documented decision: RabbitMQ stays for work queues; Kafka carries the event stream. GET /api/admin/streaming/topics and /consumer-groups.

FRONTEND OUTPUT

- Admin Streaming page: topics, partitions, events/sec, consumer lag per group.
- Live event tail with type filter; event detail viewer (JSON payload).
- Lag alert badge.

Key skills: Brokers, topics, partitions, offsets, consumer groups, retention, delivery semantics.

Done when: Platform events flow into Kafka and lag is visible per consumer group.

Phase 14: Event Streaming Pipeline

PLANNED

Goal: Build Services > Kafka > Consumers > Storage with several independent consumers.

BACKEND OUTPUT

- Independent consumer groups: Incident, Analytics, Notification, Data Pipeline. One event feeds all four, each with its own offsets and failure isolation.
- Notification consumer sends email/Slack/webhook on IncidentCreated/Resolved.
- Data Pipeline consumer writes raw events to storage (feeds Phase 15).
- Replay by resetting offsets; dead-letter topic for poison messages.
- GET/PUT /api/notifications/preferences, GET /api/notifications.

FRONTEND OUTPUT

- Notification center (bell + feed).
- Notification preferences: channel, minimum severity, subscribed services.
- Pipeline status view: state, lag, and last event time per consumer; 'one event, four consumers' demo view.

Key skills: Consumer groups, fan-out, replay, dead-letter topics, notification design.

Done when: Stopping one consumer does not affect the others, and replay reprocesses history correctly.

Phase 15: Data Lake / Data Storage Layer		PLANNED
Goal: Create a proper data platform separate from operational data.		
BACKEND OUTPUT		FRONTEND OUTPUT
• Zones: Raw (immutable) > Clean (validated, deduped, typed) > Curated (business-ready). • Store raw logs, events, incidents, deployments, and service metrics as Parquet partitioned by date/service (MinIO locally, cloud storage later). • Separate OLTP (SQL Server) from OLAP; documented trade-offs: Data Lake vs Warehouse vs Lakehouse; retention policy. • Dataset catalog metadata (name, schema, zone, size, last update); GET /api/data/datasets.		• Data Catalog page: datasets grouped by zone with schema, row count, size, last updated. • Dataset preview (first N rows) for Admin/Developer; storage growth chart.
Key skills: OLTP vs OLAP, data lake zones, Parquet, partitioning, retention.		
Done when: Raw, clean, and curated datasets exist and are browsable, with operational load untouched.		

V6: Processing, Warehouse & Orchestration		Phases 16 to 18
Version goal: Large-scale processing and a dimensional warehouse, run automatically on schedule.		
Backend release		Frontend release
PySpark jobs, star-schema warehouse with SCD2, Airflow DAGs with retries, backfill, and sensors.		Pipeline runs page, BI report explorer over the warehouse, orchestration page with trigger/backfill.
Demo scenario: Airflow runs the nightly pipeline Kafka > Spark > Warehouse; a report shows incidents per service per month.		

Phase 16: Spark Data Processing		PLANNED
Goal: Process large volumes of operational data.		
BACKEND OUTPUT		FRONTEND OUTPUT
• PySpark jobs: Raw > Clean (parse, type, dedupe) > Curated; batch first, then Structured Streaming from Kafka. • DataFrames, transformations vs actions, filters, aggregations, joins (logs x deployments x incidents), window functions (rolling error rate). • Partitioning, shuffle awareness, caching, broadcast joins, explain plans, Spark UI analysis. • Curated outputs: hourly service metrics, enriched incidents. • POST /api/admin/pipelines/{name}/run, GET runs.		• Pipeline Runs page: status, duration, rows in/out, history chart. • Preview of curated outputs (hourly error rate per service); link to Spark UI; manual trigger for Admin.
Key skills: Python, PySpark, joins, window functions, partitioning, shuffles, performance tuning.		
Done when: Spark turns raw events into curated tables and the optimization gains are measured.		

Phase 17: Data Warehouse		PLANNED
Goal: Build the analytical model.		
Backend Output		Frontend Output
- Star schema: FactIncidents, FactLogs (aggregated), FactDeployments, FactServiceMetrics. - Dimensions: DimService, DimDate, DimSeverity, DimStatus, DimEnvironment; Slowly Changing Dimension Type 2 for DimService. - Grain defined per fact table, surrogate keys, incremental Curated > Warehouse loads, analytical SQL views. - Documented star vs snowflake decision.		- BI connection (Metabase or Superset) over the warehouse. - Report Explorer: choose a measure and a dimension (e.g. incidents by service by month) with date, severity, and environment filters; saved reports.
Key skills: Star/snowflake schema, fact and dimension tables, SCD, OLTP vs OLAP.		
Done when: A business question such as 'incidents per service per month' is answered with one warehouse query.		

Phase 18: Airflow / Data Orchestration		PLANNED
Goal: Orchestrate the data pipelines.		
Backend Output		Frontend Output
- Airflow DAG: Kafka > Raw > Spark > Validation > Transformation > Warehouse > Analytics. - Dependencies, scheduling, retries with backoff, SLAs, sensors, backfill, failure callbacks. - Idempotent, re-runnable tasks per date partition; DAG run metrics; GET /api/admin/pipelines (proxy of the Airflow REST API).		- Orchestration page: DAG list, last/next run, duration, failure count; run-history grid. - Admin actions: trigger, backfill a date range (with confirmation), open Airflow UI; failure alert banner.
Key skills: DAGs, tasks, scheduling, sensors, backfill, failure handling, monitoring.		
Done when: The whole pipeline runs on schedule, recovers from failures, and can be backfilled.		

V7: Trusted Data, Analytics & Advanced AI		Phases 19 to 21
Version goal: Quality-checked data powers SRE metrics and history-aware AI.		
Backend release	Frontend release	
Data quality framework with quarantine and alerts, analytics API (MTTR, MTBF, availability...), vector store with similar-incident retrieval.	Data Quality dashboard, analytics dashboard with export, similar-incidents panel, confirmed-root-cause capture, AI context viewer.	
Demo scenario: A bad batch is quarantined and alerted; MTTR trends render; a new incident shows similar past incidents and their causes.		

Phase 19: Data Quality		PLANNED
Goal: Ensure pipelines produce trustworthy data, not just running data.		
BACKEND OUTPUT	FRONTEND OUTPUT	
- Checks (Great Expectations/Soda/custom): schema, nulls, duplicates, data types, ranges, missing data, freshness. - Volume reconciliation: expected 1,000,000 logs vs received 700,000 triggers a Data Quality alert. - Bad records quarantined to a rejected zone with a reason; results stored in DataQualityRun / DataQualityCheck. - Alerts delivered through the Notification consumer; GET /api/data-quality/summary and /checks.	- Data Quality dashboard: per-dataset scorecard (pass rate, freshness) with trend. - Failed-check drill-down with sample bad records; alert list with acknowledge. - Expected vs received volume gauge.	
Key skills: Validation, freshness, reconciliation, quarantine, pipeline monitoring. Done when: A deliberately corrupted or incomplete batch is caught, quarantined, and alerted.		

Phase 20: Analytics Layer		PLANNED
Goal: Create useful operational metrics.		
BACKEND OUTPUT	FRONTEND OUTPUT	
- Metrics: MTTR, MTBF, incident frequency, severity distribution, error rate, service availability, most failing services, deployment failure rate, incident resolution time. - Analytics API: GET /api/analytics/mttr, /availability, /incident-frequency, ... (from, to, serviceId); pre-aggregated and cached in Redis. - Pipeline: Operational > Kafka > Spark > Warehouse > Analytics; written metric definitions and formulas.	- Analytics dashboard: MTTR/MTBF trends, weekly incident frequency, severity distribution, availability per service, top failing services, deployment failure rate. - Date range, service, and environment filters; period comparison; CSV/PNG export; definition tooltips.	
Key skills: KPI design, SRE metrics, aggregation, API design for analytics. Done when: Leadership-ready metrics are computed from the warehouse and match the definitions.		

Phase 21: Advanced AI / RAG		PLANNED
Goal: Improve root cause analysis with historical knowledge.		
BACKEND OUTPUT	FRONTEND OUTPUT	
- Retrieval: current incident > logs > timeline > deployments > similar historical incidents > context > LLM > RCA. - Resolved incidents are embedded and indexed in a vector store (pgvector/Qdrant/Azure AI Search) with metadata filters (service, severity). - Resolve now captures a Confirmed Root Cause, feeding the knowledge base. - Citations for every evidence item; evaluation set of past incidents with known causes to measure accuracy; GET /api/incidents/{id}/similar.	- Similar Incidents panel: similarity score, previous root cause, resolution. - Context viewer ('what the AI saw'); clickable evidence citations. - Resolve dialog requires the Confirmed Root Cause; compare AI vs confirmed result.	
Key skills: Embeddings, vector search, hybrid retrieval, evaluation, citations. Done when: New incidents get analyses that reference similar past incidents, with measurable accuracy.		

V8: Secure, Tested & Proven

Phases 22 to 24

Version goal: The platform is hardened, fully tested, and load-validated.

Backend release

Secrets management, token rotation, OWASP checklist, audit logs, full test pyramid with E2E flow, k6 load tests with baselines.

Frontend release

Secure session handling, CSP, session and audit pages, component and Playwright E2E suites, Web Vitals budget.

Demo scenario: CI runs the whole test suite; a load report shows behavior at 100, 500, and 1000 req/s.

Phase 22: Security Hardening

PLANNED

Goal: Make the platform secure by design.

BACKEND OUTPUT

- Secrets management (user-secrets/Key Vault); secure per-environment configuration.
- Password hashing review and lockout; JWT hardening (short expiry, key rotation, issuer/audience validation); refresh token rotation with reuse detection.
- Policy review, input validation, rate limiting on auth endpoints, CORS allow-list, HTTPS/HSTS, security headers.
- Audit log and GET /api/admin/audit-logs; OWASP API Security Top 10 checklist; dependency scanning.

FRONTEND OUTPUT

- Secure session handling: in-memory access token + httpOnly refresh cookie, silent rotation.
- Content-Security-Policy; XSS-safe rendering of logs and messages.
- Session management page (active sessions, revoke); Admin Audit Log viewer; optional MFA setup; lockout and rate-limit messages.

Key skills: OWASP API Security, JWT security, secrets, CORS, headers, audit logging.

Done when: The OWASP checklist passes and a stolen refresh token is detected and revoked.

Phase 23: Testing Strategy

PLANNED

Goal: Prove the system works, not just the units.

BACKEND OUTPUT

- Test pyramid: unit (state machine, correlation, validators), integration (Testcontainers for SQL, Redis, RabbitMQ, Kafka), API contract, database, and messaging tests (duplicates, DLQ).
- End-to-end flow test: POST Log > Database > RabbitMQ > Worker > Incident > Timeline > Notification.
- Phase 11 failure drills automated; coverage gate in CI; test data builders.

FRONTEND OUTPUT

- Component and service tests (Jest or Karma/Jasmine with Angular Testing Library); HttpTestingController / mock server for API edge cases; accessibility checks (axe).
- Playwright E2E: login > create incident > transition > timeline > AI panel.
- Contract tests against the OpenAPI spec.

Key skills: Test pyramid, Testcontainers, contract tests, E2E, failure testing.

Done when: One command runs the full suite and the end-to-end flow is green in CI.

Phase 24: Performance & Load Testing		PLANNED
Goal: Measure the system under realistic load.		
BACKEND OUTPUT • k6 load tests at 100, 500, and 1000 requests/sec on POST /logs, GET /incidents, and dashboard statistics. • Measure latency (p50/p95/p99), throughput, CPU, memory, database load, Redis hit rate, queue length, Kafka consumer lag. • Bottleneck analysis and fixes; performance baseline with regression thresholds in CI; report with graphs.		FRONTEND OUTPUT • Web Vitals / Lighthouse budget, lazy-loaded feature routes, OnPush change detection, CDK virtual scroll, debounced search. • Grafana load-test dashboard embedded in Admin; UI behavior under load (loading states, backpressure) verified.
Key skills: Load testing, capacity planning, bottleneck analysis, SLO thinking. Done when: A published report states what the system handles at each load level and where it breaks.		

V9: Production Release (1.0)		Phases 25 to 27
Version goal: Containerized, deployed to the cloud, and documented as a complete production architecture.		
Backend release Docker Compose for the full stack, CI/CD, Azure deployment from IaC, architecture docs, runbooks, DR drill.		Frontend release Containerized Angular frontend on cloud hosting with CDN and HTTPS, unified console with dark mode and demo mode.
Demo scenario: Run the full seeded scenario live in the cloud: deploy > spike > auto incident > AI RCA > resolve > analytics.		

Phase 25: Docker & Deployment		PLANNED
Goal: Containerize everything and automate delivery.		
BACKEND OUTPUT • Containers for API, Worker, SQL Server, Redis, RabbitMQ, Kafka, Spark, Airflow (plus MinIO, warehouse, OTel collector). • Docker Compose profiles (core / data / full) so a laptop can run subsets. • CI/CD (GitHub Actions or Azure DevOps): build, test, scan, push images, deploy; environment-based config and migrations.		FRONTEND OUTPUT • Frontend Docker image (nginx) with runtime API URL config. • CI: lint, type-check, test, build, image; optional per-PR preview deployments. • docker compose up serves the whole system with the UI at localhost.
Key skills: Docker, Compose, CI/CD, configuration management. Done when: A clean machine runs the entire platform with one command, and every merge is built and tested automatically.		

Phase 26: Cloud		PLANNED
Goal: Deploy to one cloud (Azure suggested because of .NET) and learn cloud architecture, not service names.		
BACKEND OUTPUT		FRONTEND OUTPUT
• Compute (App Service/Container Apps/AKS), storage (Blob/ADLS), networking (VNet, private endpoints), IAM (Managed Identity, RBAC). • Managed databases (Azure SQL, Azure Cache for Redis), queues/streams (Service Bus or managed RabbitMQ, Event Hubs for Kafka), monitoring (Azure Monitor/App Insights), secrets (Key Vault). • Autoscaling, load balancing (App Gateway/Front Door); Infrastructure as Code (Bicep/Terraform); dev/staging/prod environments; cost notes.		• Static hosting (Static Web Apps or Blob + CDN), custom domain, HTTPS. • Environment-specific config, CDN cache headers, frontend telemetry to App Insights, staged release pipeline.
Key skills: Compute, storage, networking, IAM, managed services, scaling, IaC. Done when: The platform runs in the cloud from IaC with staging and production environments.		

Phase 27: Production Architecture		PLANNED
Goal: Reach the final target architecture.		
BACKEND OUTPUT		FRONTEND OUTPUT
• Final flow: Clients > API Gateway > Incident API + Auth > SQL Server > Redis / RabbitMQ > Workers > Kafka > Spark / Consumers > Data Lake / Warehouse > Analytics; Logs + Incidents + History > RAG > LLM > Root Cause Analysis; OpenTelemetry across everything. • Architecture docs: C4 diagrams, ADRs, runbooks, SLOs; backup/restore and disaster-recovery drill. • Seed-data generator and full end-to-end demo script; final scale and cost review.		• Unified Incident Intelligence Console with sections: Incidents, Services, Logs, Dashboard, Analytics, AI Insights, Data Quality, Pipelines, Admin. • Consistent Angular Material theme and shared component library, dark mode, accessibility pass. • Demo mode: deploy > error spike > auto incident > AI root cause > resolve > analytics update; in-app architecture page.
Key skills: System design, documentation, operations, end-to-end ownership. Done when: The complete scenario runs live from a seeded environment and the production-readiness checklist is signed off.		

Delivery Philosophy and Milestones

No phase is a fixed-duration sprint. Each phase is a milestone reached when its backend and frontend outputs work end to end. Each version is released only when its demo scenario can be run live.

Version	Milestone statement
MVP	I can build and secure a complete application with a business workflow.
V2	I can work with real operational data, automate detection, and make it fast.
V3	I understand message queues, AI integration, and observability.
V4	I can design for failure and understand what happens under EF Core.
V5	I understand event streaming and multiple independent consumers.
V6	I can process large data and model it for analytics, orchestrated automatically.
V7	I can guarantee data quality and build analytics and history-aware AI.
V8	I can secure, test, and load-test a system.
V9	I can deploy and operate a production architecture end to end.

Immediate next step: finish Phase 5 (Logs & Correlation, backend and frontend outputs), then proceed to Phase 6 to complete V2.